



УЧЕБНАЯ ПЛАТФОРМА

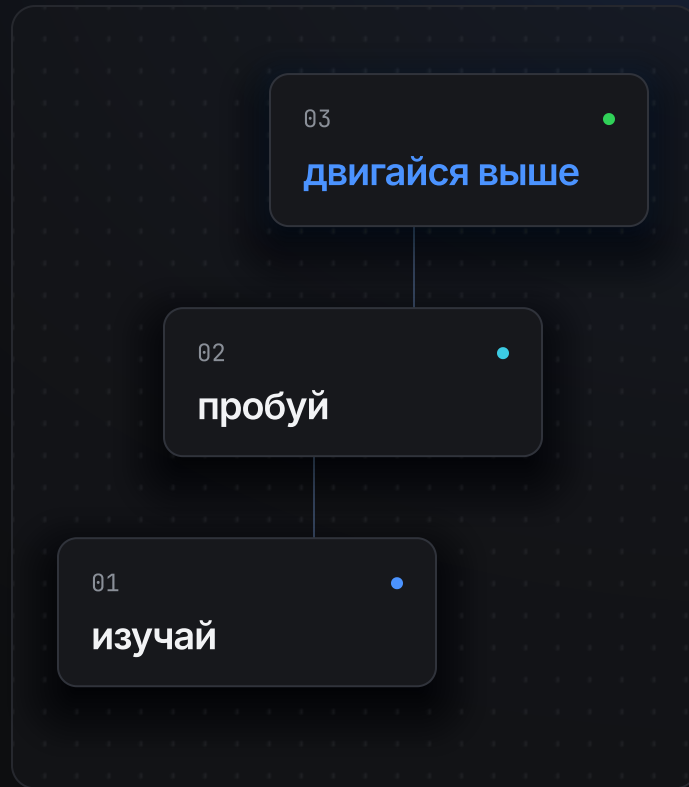
ЗНАНИЯ СТАНОВЯТСЯ практикой.

От первого урока до работающего кода.

• курсы

• практика

• AI-помощь



проблема



обучение построено вокруг слайдов и конспектов

01 · на паре

пара уходит на переписывание
лекции

02 · практика

код пишется на своих компьютерах —
проверить его сложно

03 · помощь

вопрос возникает вечером над
домашкой — а спросить некого

04 · подготовка

преподаватель готовит материалы
вручную

было → стало



что меняет платформа

было

стало

презентация на паре



интерактивный урок с кодом, доступный всегда

конспект в тетрадь



материал уже на платформе, пара — на практику

код на слайде



код запускается и проверяется прямо в браузере

вопросы только на паре



AI-тьютор по материалу урока 24/7

проверка раз в семестр



тесты и задачи с автопроверкой после урока

всё вручную



AI-ассистент и AI-агенты через MCP помогают авторам



четыре роли на одной платформе

студент

- курсы, уроки и тесты в одном месте
- задачи с запуском кода
- проекты в команде и публичный профиль

преподаватель

- треки, курсы, модули и уроки
- тесты и задачи с тест-кейсами
- правки от AI-ассистента

администратор

- назначать роли
- управлять доступом
- публиковать новости

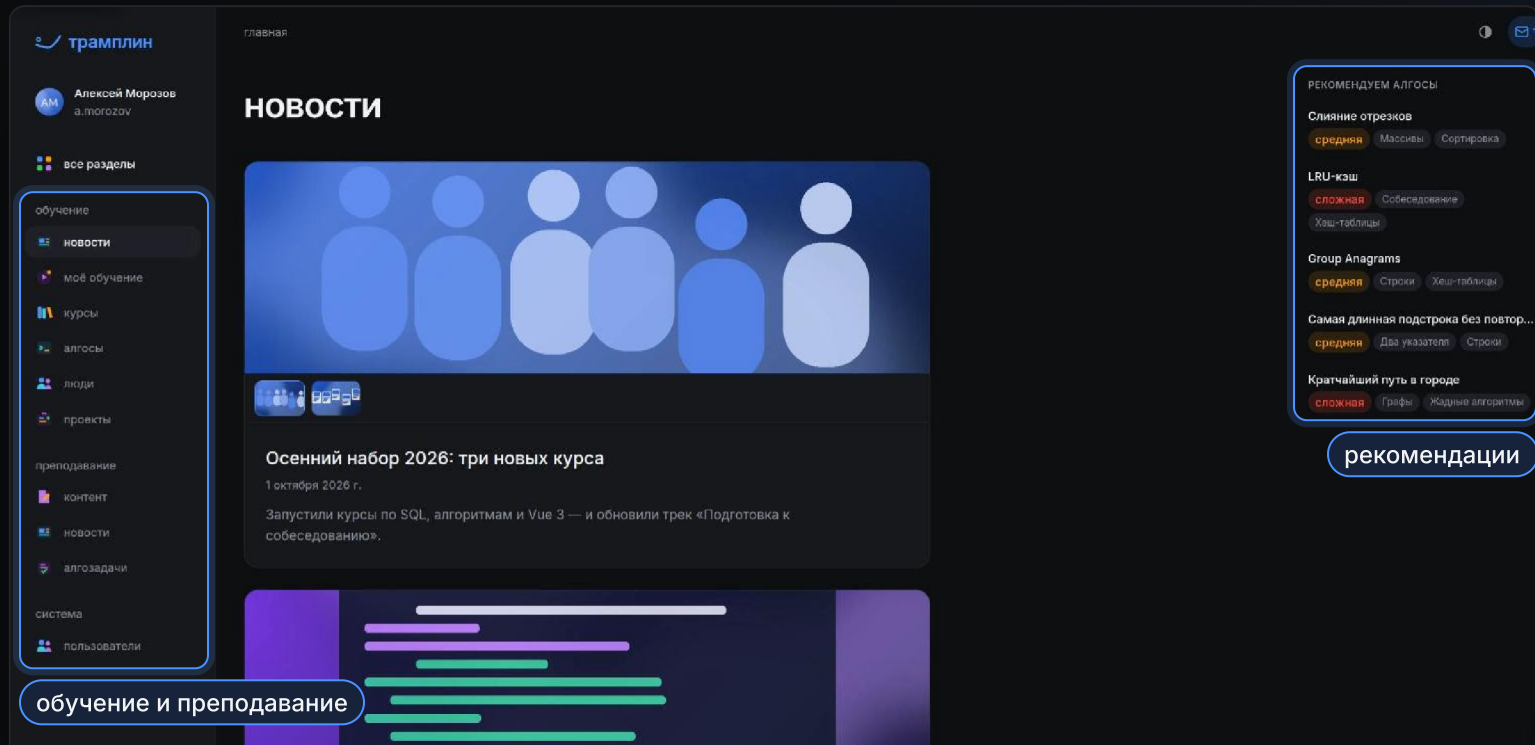
• AI-агент

- читает курс
- готовит план правок
- создаёт черновики
- не публикует и не удаляет



ОДИН ВХОД ВО ВСЁ ОБУЧЕНИЕ

- разделы «обучение» и «преподавание» в одном сайдбаре
- новости платформы на главной
- рекомендации задач по интересам и прогрессу



обучение и преподавание

маршрут курса из СВЯЗАННЫХ модулей

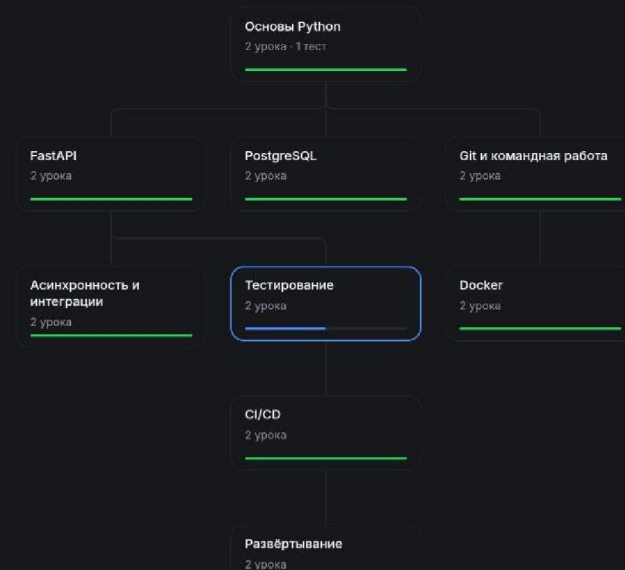
- модули складываются в маршрут курса
- связь — рекомендованный порядок, а не замок
- прогресс виден по каждому модулю

Полный путь Junior-разработчика: Python, FastAPI, PostgreSQL, Docker, тесты, CI/CD и мониторинг.

18 из 20 уроков

карта курса

Связи — рекомендованный порядок, а не замок: открыть можно любой модуль.





уроки и тесты с учётом прогресса

- Markdown с подсветкой кода
- 6 типов вопросов: один и несколько ответов, текст, сопоставление, группировка, файл
- результат учитывается в прогрессе

[главная](#) / [курсы](#) / [http и rest api](#) / [как устроен http](#) / [запрос и ответ](#)

Запрос и ответ

15 мин

Запрос и ответ

HTTP — это протокол вопросов и ответов. Клиент присылает **запрос**, сервер возвращает **ответ**. Больше в протоколе ничего нет: ни соединения «на память», ни очереди сообщений, ни подписки на события.

Из чего состоит запрос

```
GET /api/v1/courses?limit=20 HTTP/1.1
Host: tramlinn.tech
Accept: application/json
Authorization: Bearer eyJhbGciOi...
```

Первая строка — метод, путь и версия протокола. Дальше идут заголовки: по одному на строку, **Имя: значение**. После пустой строки может идти тело — у **GET** его обычно нет, у **POST** там лежат данные.

[главная](#) / [курсы](#) / [http и rest api](#) / [как устроен http](#) / [проверка: основы http](#)

Проверка: основы HTTP

лучший результат: 3 из 3

- Какой код вернуть на успешный 'DELETE', если тело ответа не нужно?
 - ☐ 200 OK
 - ☒ 204 No Content
 - ☐ 404 Not Found
 - ☐ 410 Gone
- Отметьте идиempотентные методы.
 - ☒ GET
 - ☐ POST
 - ☒ PUT
 - ☒ DELETE
 - ☐ PATCH
- Клиент прислал верный токен, но не имеет прав на операцию. Какой код?
 - ☐ 400
 - ☐ 401

обучение · AI-тьютор

AI-тьютор по материалу урока

01 · контекст

объясняет материал конкретного урока

02 · без готовых ответов

в тестах объясняет принцип на похожем примере, а не даёт ответ

03 · задачи

в алгоритмах подсказывает идею и находит ошибку в коде — решение пишет студент

• AI-тьютор

урок: запрос и ответ

Почему у GET-запроса обычно нет тела?

ассистент

GET только получает данные: всё нужное передаётся в пути и параметрах.

```
GET /api/v1/courses?limit=20
```

Тело нужно, когда данные отправляются на сервер, — как в POST.

квота запросов

спросите по уроку...

спросить



алгоритмические задачи прямо в браузере

- условие, редактор и запуск на одном экране
- 8 языков: C++, Python, Go, Java и другие; свой ввод и тест-кейсы
- свободная практика или собеседование на время

главная | алгосы

← ко всем задачам

Слияние отрезков

средняя Массивы Сортировка

1000 мс · 256 МБ

Дано n отрезков $[l, r]$. Объедините пересекающиеся и выведите результат в порядке возрастания левых концов.

Формат ввода

n , затем n пар чисел.

Формат вывода

Количество отрезков, затем сами отрезки по одному в строке.

▼ пример 1

ввод	вывод
4	3
1 3	1 6
2 6	8 10
8 10	15 18
15 18	

C++ ▾ 8 языков

решить заново

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7     // ваше решение
8     return 0;
9 }
10
```

► свой ввод для запуска

запустить

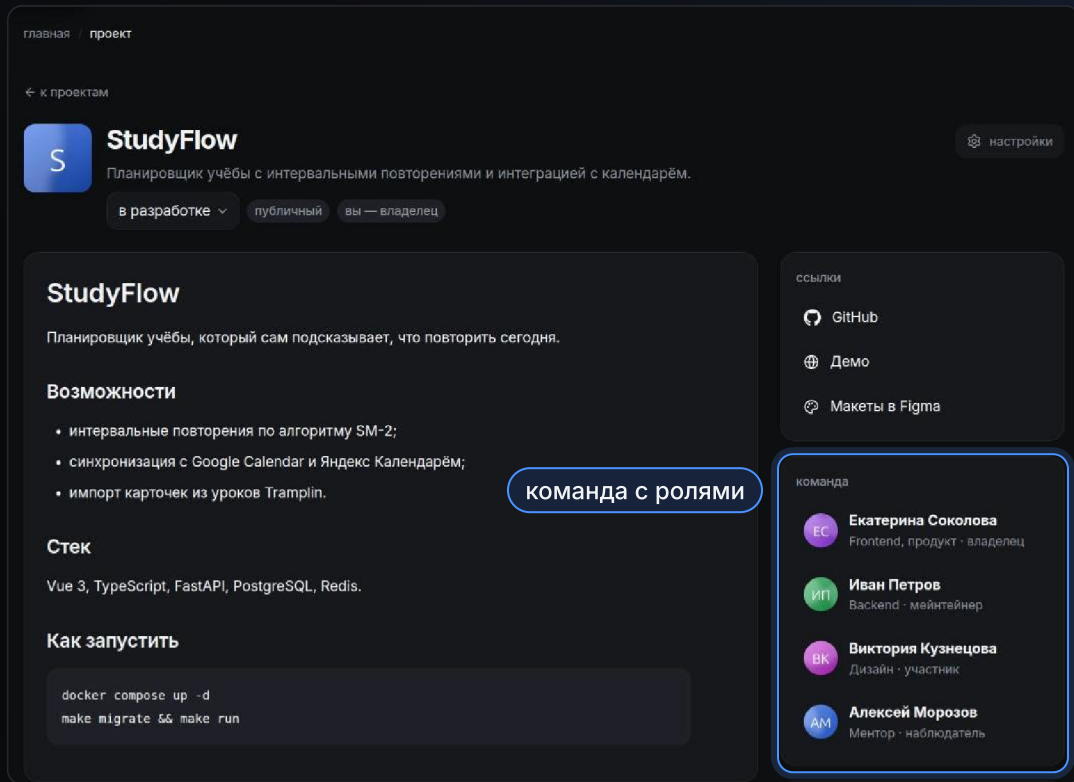
отправить

запуск в изолированной песочнице

проекты

проекты студентов и публичные профили

- README, команда с ролями и статус проекта
- ссылки на GitHub, демо и Figma
- публичные профили с прогрессом и приглашения в команду





инструменты преподавателя

- треки → курсы → модули → уроки
тесты и задачи с тест-кейсами
- Markdown с живым предпросмотром,
черновики и публикация
- AI-ассистент предлагает правки —
автор принимает или отклоняет

[главная](#) [контент / урок](#)

Запрос и ответ

опубликован ← к списку

markdown

1 карточка предпросмотр

Запрос и ответ

HTTP — это протокол вопросов и ответов. Клиент присылает ****запрос****, сервер возвращает ****ответ****. Больше в протоколе ничего нет: ни соединения «на память», ни очереди сообщений, ни подписки на события.

Из чего состоит запрос

```
```\nhttp\nGET /api/v1/courses?limit=20 HTTP/1.1\nHost: tramlinn.tech\nAccept: application/json\nAuthorization: Bearer eyJhbGciOi...\n```
```

Первая строка — метод, путь и версия протокола. Дальше идут заголовки: по одному на строку, **Имя: значение**. После пустой строки может идти тело — у **GET** его обычно нет, у **POST** там лежат данные.

Ответ устроен так же, только вместо метода и пути в первой строке стоит код состояния:

```
```\nhttp\nHTTP/1.1 200 OK\nContent-Type: application/json\nContent-Length: 42\n```
```

Запрос и ответ

HTTP — это протокол вопросов и ответов. Клиент присылает **запрос**, сервер возвращает **ответ**. Больше в протоколе ничего нет: ни соединения «на память», ни очереди сообщений, ни подписки на события.

Из чего состоит запрос

```
GET /api/v1/courses?limit=20 HTTP/1.1\nHost: tramlinn.tech\nAccept: application/json\nAuthorization: Bearer eyJhbGciOi...
```

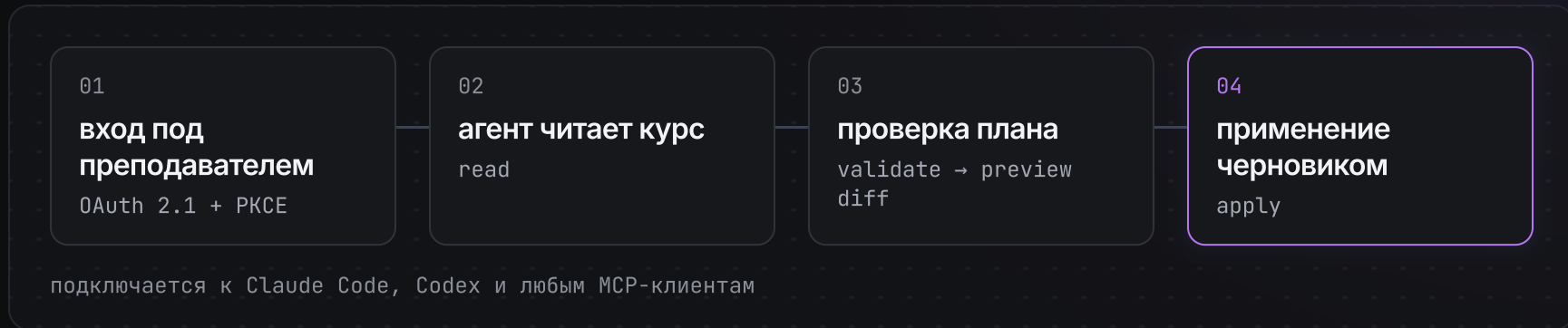
Первая строка — метод, путь и версия протокола. Дальше идут заголовки: по одному на строку, **Имя: значение**. После пустой строки может идти тело — у **GET** его обычно нет, у **POST** там лежат данные.

Ответ устроен так же, только вместо метода и пути в первой строке стоит код состояния:

```
HTTP/1.1 200 OK\nContent-Type: application/json\nContent-Length: 42
```



наполнение курса через AI-агента



**AI предлагает —
человек решает.**

- ничего не удаляется
- всё остаётся черновиком
- ничего не публикуется само
- планы идемпотентны

система

ВХОД И ИНТЕГРАЦИИ

вход

GitHub, Яндекс ID, GitLab или код на почту — без паролей

аккаунт

несколько способов входа
привязываются к одному аккаунту

интеграции

МСР для AI-агентов; ссылки проектов
на GitHub, GitLab, Figma и демо

вход в один клик

 трамплин

Войдите, чтобы продолжить

 GitHub

 Яндекс ID

 GitLab

или по почте

почта

получить код

под капотом

архитектура из пяти сервисов

frontend

Vue 3 · TypeScript · Vite · Pinia

backend

FastAPI · PostgreSQL · pgvector · Redis

email-service

FastStream · RabbitMQ

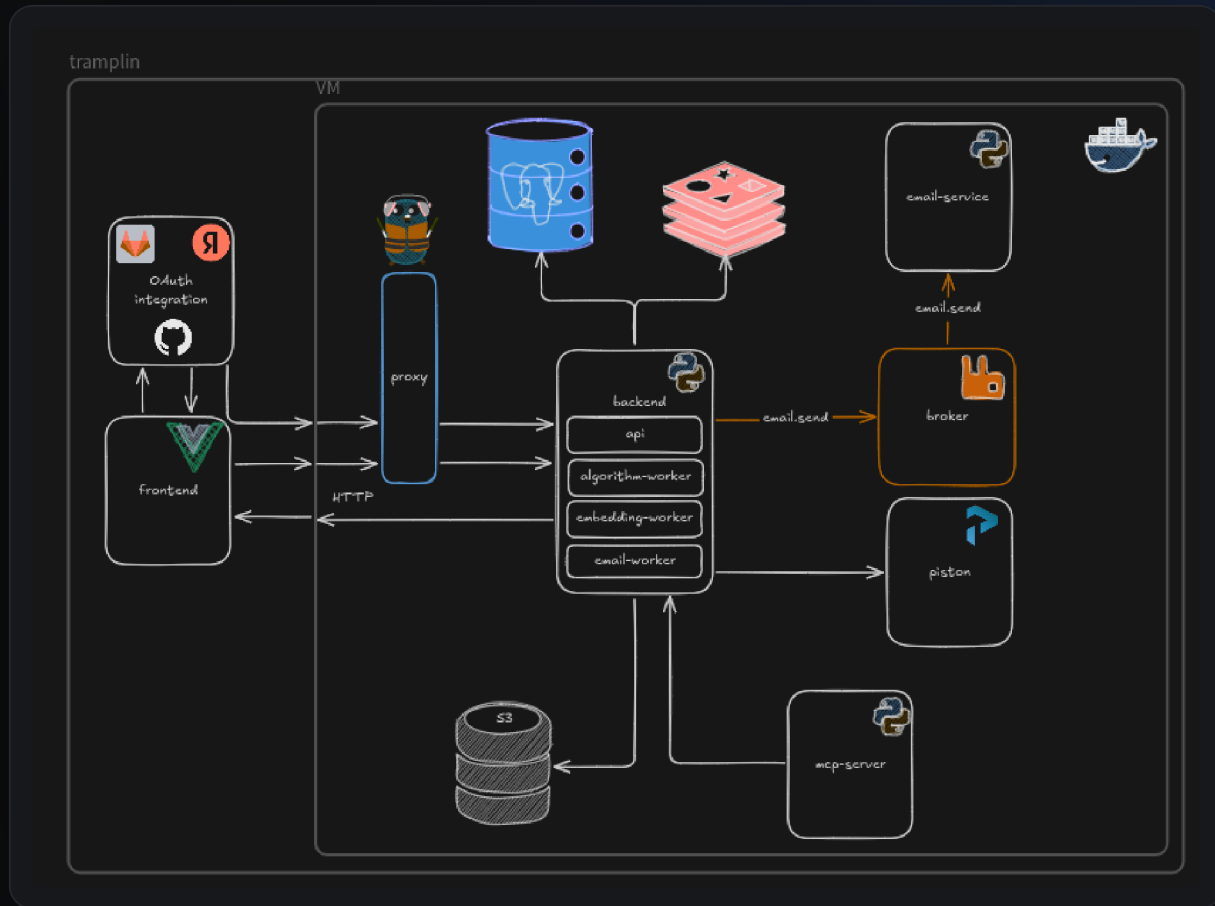
mcp-server

FastMCP · OAuth 2.1 + PKCE

infra

Docker Compose · Caddy · Piston · Grafana

- backend — единственный владелец данных
- запуск кода изолирован и не видит базу





инженерное качество проекта

`pytest --cov`

≥ 90 %

покрытие тестами в Python-сервисах

`mypy --strict`

строгая проверка типов

ESLint · Prettier ·
Vitest

линтинг, форматирование и тесты
фронтенда

CI/CD

security-сканирование и
immutable-образы

Prometheus · Loki ·
Grafana

мониторинг: метрики, логи и
дашборды



ценность и целевые метрики пилота

студенты

один вход во всё обучение и практика с запуском кода

мгновенная обратная связь и AI-тьютор

преподаватели

уроки, тесты и задачи создаются быстрее

автопроверка снимает рутину проверки

учебное заведение

своя площадка: контент и данные у себя

практика для всех групп и стоимость AI под контролем

целевые метрики пилота

• гипотезы — уточняются после пилота

≥ 60 %

WAU от числа зачисленных

≥ 50 %

завершённых уроков

≥ 3

задачи на студента в неделю

−40 %

времени на подготовку урока

≥ 50 %

принятых предложений AI-ассистента



ЗНАНИЯ СТАНОВЯТСЯ ПРАКТИКОЙ.

tramplinn.tech

